



Vize

Algoritmaların Özellikleri

1. Sonluluk:

- a. Algoritma bir noktada durmak zorundadır, sonsuza kadar gidemez.

2. Belirlilik:

- a. Her adım açık ve net, yorum gerektirmeyecek şekilde olmalıdır.

3. Girdi / Çıktı:

- a. Algoritmalar sıfır veya daha fazla girdi alabilirler
- b. En az bir çıktı üretmek zorundadırlar.

4. Etkinlik:

- a. Her adım uygulanabilir olmalıdır.

Asimtotik Notasyonlar

Big-O Notasyonu

- Algoritmanın tavan büyüme eğrisini gösterir.

$$0 < T(n) \leq c \cdot f(n)$$

T(n): Senin yazdığın algoritmanın gerçek çalışma süresi.

f(n): Kıyaslama yaptığımız standart, basit bir matematiksel fonksiyon.

c (Sabit Çarpan): Donanım hızı, programlama dili gibi ufak detayları göz ardı etmemizi sağlayan pozitif bir sayı. Biz asıl odaklanmamız gereken "büyüme eğilimine" bakarız.

- $T(n)$, $f(n)$ 'den **daha hızlı büyümez**. En kötü durumda bile $f(n)$ mertebesinde.

Big-O Bulmak: Örnek 1

Adım 1: $f(n)$ 'i Seçiyoruz : Tahminimiz, bu fonksiyonun doğrusal (n) olduğu. O yüzden **$f(n) = n$** seçiyoruz.

Adım 2: Eşitsizliğimizi Yazıyoruz : Tanıma göre, öyle bir c bulmalıyız ki, aşağıdaki eşitsizlik bir yerden sonra hep sağlansın: $3n + 2 \leq c * n$

Adım 3: c Sabitini Bulmaya Çalışıyoruz : Şimdi bu eşitsizliği düzenleyelim. Her iki taraftan $3n$ çıkaralım: $2 \leq c*n - 3n \rightarrow 2 \leq n*(c - 3)$

Bu eşitsizliğin sağlanması için iki şart var: $(c - 3)$ **pozitif olmalı**, yani $c > 3$ olmalı. Çünkü negatif olursa, n büyüdükçe sağ taraf küçülür ve eşitsizlik hiç sağlanmaz.

n yeterince büyük olmalı.

Adım 4: Mantıklı bir c Değeri Seçiyoruz : Diyelim ki $c = 4$ seçtik. (Neden 4? Çünkü 3'ten büyük en küçük tam sayı, işimizi kolaylaştırır). Denklemde yerine koyalım:

$$3n + 2 \leq 4n \rightarrow 2 \leq 4n - 3n \rightarrow 2 \leq n$$

Adım 5: n_0 'ı (Eşiği) Buluyoruz : Bulduğumuz sonuç: $n \geq 2$. Yani ne zaman n , 2'den büyük veya eşit olursa, bu eşitsizlik kesinlikle doğrudur.

İşte bu eşik değerine **$n_0 = 2$** diyoruz.

Adım 6: Sonuç : $c = 4$ ve $n_0 = 2$ seçtiğimizde, $n \geq 2$ için **$3n + 2 \leq 4n$** her zaman doğrudur. Bu da demek oluyor ki,

$T(n) = 3n + 2$ fonksiyonu, $O(n)$ sınıfına aittir.

- Ama $3n+2$, n 'den her zaman büyük değil mi? Mesela $n=1$ için $5 > 4$?"
 - **n_0** devreye burada giriyor.
 - **$n=1$** için: $T(1)=5$, $c*n = 4*1 = 4$. $5 \leq 4$ mü? **Hayır**. (Bu yüzden eşik değeri 1 değil).
 - **$n=2$** için: $T(2)=8$, $c*n = 4*2 = 8$. $8 \leq 8$ mü? **Evet**.
 - **$n=3$** için: $T(3)=11$, $c*n = 12$. $11 \leq 12$ mi? **Evet**.
 - **$n=100$** için: $T(100)=302$, $c*n = 400$. $302 \leq 400$ mü? **Evet**.
 - **$n=2$ eşiğini geçtikten sonra**, $c*n$ (yani $4n$), her zaman $3n+2$ 'den büyük.
- **Neden $c=4$ Seçtik?**
 - Başka c değerleri de seçebilirdik, önemli olan var olduğunu göstermek.
 - Eğer $c = 3.5$ seçseydik: $3n+2 \leq 3.5n \Rightarrow 2 \leq 0.5n \Rightarrow n \geq 4$ olurdu. Bu durumda $n_0=4$ olurdu. Bu da geçerli bir çözümdür.
 - Eğer $c = 10$ seçseydik: $3n+2 \leq 10n \Rightarrow 2 \leq 7n \Rightarrow n \geq 1$ olurdu. $n_0=1$ olurdu.
 - **Özetle:** Büyük-O, bize bir **üst sınır** verir. 4, 5, 10, 100 hepsi üst sınır olabilir. Bizim örnekte pratik olması için 4'ü seçtik.

Big-O Bulmak: Örnek 2

Aşağıda verilen zaman karmaşıklığı fonksiyonu için **Büyük-O** notasyonunu bulunuz ve **küçük n değerleri için test ederek** gösteriniz.

$$T(n) = 5n^2 + 3n + 1$$

- **Adım 1: $f(n)$ 'i Seçiyoruz:** Tahminimiz, bu fonksiyonun karesel (yani n^2) olduğu. Çünkü bir polinomda büyüme hızını her zaman derecesi en büyük olan terim belirler. O yüzden $f(n) = n^2$ seçiyoruz.
- **Adım 2: Eşitsizliğimizi Yazıyoruz:** Tanıma göre, öyle bir c bulmalıyız ki, aşağıdaki eşitsizlik bir yerden sonra hep sağlansın:

$$5n^2 + 3n + 1 \leq c \cdot n^2$$
- **Adım 3: c Sabitini Bulmaya Çalışıyoruz:** Şimdi bu eşitsizliği düzenleyelim. Her iki taraftan $5n^2$ çıkaralım:

$$3n + 1 \leq c \cdot n^2 - 5n^2 \implies 3n + 1 \leq n^2(c - 5)$$
- Bu eşitsizliğin her zaman sağlanması için sağ tarafın pozitif kalması gerekir. Yani $(c - 5)$ **pozitif olmalı**, dolayısıyla $c > 5$ olmalı.
- **Adım 4: Mantıklı bir c Değeri Seçiyoruz:** Diyelim ki $c = 9$ seçtik. (Neden 9? Polinomlarda pratik bir yöntem olarak tüm katsayıları toplamak en kolay c değerini verir: $5 + 3 + 1 = 9$. Sen istersen 10 da seçebilirsin, sonuç değişmez). Denklemden yerine koyalım:

$$5n^2 + 3n + 1 \leq 9n^2$$
- **Küçük n Değerleri İçin Test Ediyoruz (Sorunun Ekstra İsteği):** Bakalım seçtiğimiz $c = 9$ değeri işe yarıyor mu?
 - $n = 1$ için: $5(1)^2 + 3(1) + 1 \leq 9(1)^2 \implies 9 \leq 9$ (Doğru, eşitsizlik sağlandı)
 - $n = 2$ için: $5(2)^2 + 3(2) + 1 \leq 9(2)^2 \implies 20 + 6 + 1 \leq 36 \implies 27 \leq 36$ (Doğru)
 - $n = 3$ için: $5(3)^2 + 3(3) + 1 \leq 9(3)^2 \implies 45 + 9 + 1 \leq 81 \implies 55 \leq 81$ (Doğru)
- **Adım 5: n_0 'ı (Eşiği) Buluyoruz:** Testlerimizde gördüğümüz gibi, n yerine 1 koyduğumuz andan itibaren eşitsizlik hep doğru çıkıyor. İşte bu eşik değerine $n_0 = 1$ diyoruz.
- **Adım 6: Sonuç:** $c = 9$ ve $n_0 = 1$ seçtiğimizde, $n \geq 1$ için $5n^2 + 3n + 1 \leq 9n^2$ her zaman doğrudur. Bu da demek oluyor ki;

$$T(n) = 5n^2 + 3n + 1$$
 fonksiyonu, $O(n^2)$ sınıfına aittir.

Big-Omega $\Omega()$

- Alt sınırı (Best Case) belirtir.

- Algoritma bundan daha yavaş büyüyemez.

$$0 \leq c \cdot f(n) \leq T(n)$$

- **Örnek : $T(n) = 3n + 2$ fonksiyonu $\Omega(n)$ midir?**
- $3n + 2 \geq c \cdot n$ eşitsizliğini sağlayan c ve n_0 bulalım.
- $c = 1$ seçelim. $3n + 2 \geq n \Rightarrow 2n + 2 \geq 0$ her $n \geq 0$ için doğrudur.
- O halde $T(n) = 3n + 2 = \Omega(n)$ 'dir.
- **Öneri:** c 'yi, $T(n)$ 'deki en büyük dereceli terimin katsayısından küçük veya eşit, fakat pozitif bir sabit seç. Lineer fonksiyonlar için genelde katsayının yarısı veya aynısı iş görür.

Big-Teta $\Theta()$

- Bir algoritmanın kesin sınırını belirtir. Algoritmanın tam olarak $f(n)$ mertebesinde büyüdüğünü ifade eder.
- **$T(n)$ hem $O(f(n))$ hem de $\Omega(f(n))$ ise $\Theta(f(n))$ 'dir.**

$$0 \leq c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n)$$

Bu eşitsizlik diyor ki: Veri boyutu belli bir eşiği (n_0) geçtikten sonra, senin algoritmanın çalışma süresi ($T(n)$) adeta bir sandviç gibi iki değer arasında sıkışıp kalır.

- **Alt sınır ($c_1 \cdot f(n)$):** Algoritma bu değerden daha hızlı (daha kısa sürede) çalışamaz.
- **Üst sınır ($c_2 \cdot f(n)$):** Algoritma bu değerden daha yavaş (daha uzun sürede) çalışamaz.

Büyüme Oranlarının Sıralanması

Sabit: 1

Logaritmik: $\log n$

Doğrusal: n

$n \log n$

Polinom: $n^2, n^3, \dots$

Üstel: $2^n, 3^n, \dots$

Faktöriyel: $n!$

Önemli not: $\log n$ tabanı (2, 10, e) asimptotik sınıfta fark yaratmaz

Asimptotik Analiz Teknikleri

- Aslında Big-O değeri $T(n)$ 'den büyük olduğu sürece herhangi bir $f(n)$ olabilir. Bu da demek oluyor ki herhangi bir algoritmanın aslında sonsuz adet Big-O'su vardır. Ancak $T(n)$ 'den çok büyük olan bir değer bizim hiçbir işimize yaramayacağı için genelde en dar aralıktaki Big-O'yu ararız. Bu da polinomlarda Big-O'nun Big- Ω ile aynı çıkmasına sebebiyet verir.
- Polinomlarda her zaman en yüksek dereceli terim baskındır:

$$7n^3 + 2n^2 + 100n + 1 = \Theta(n^3)$$

- Çarpım Kuralı:

$$O(f(n)) * O(g(n)) = O(f(n)g(n))$$

- İç içe döngüler çoğu zaman buraya düşer.
- Toplam Kuralı:

$$O(f(n) + g(n)) = O(\max(f(n), g(n)))$$

- Büyük olan baskın olur.

- Logaritmik ifadelerde taban fark etmez.
- $n \log n$ ve $\log(n!)$ sezgisel olarak birbirine eşittir.
- Gauss Toplama Formülü:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

- Logaritmada Toplama Çıkartma:

$$\log n + \log m = \log(n * m)$$

$$\log n - \log m = \log\left(\frac{n}{m}\right)$$

Rekürsif Algoritmalar

Rekürsif algoritmalar, kendilerini çağırma şekillerine göre 2 ana kategoriye ayrılır.

1. **Doğrudan Rekürsif:** Bir fonksiyonun, kendi gövdesi içinde doğrudan kendisini çağırmasıdır . En yaygın rekürsif türüdür.
2. **Dolaylı Rekürsif:** Bir fonksiyonun (A diyelim) başka bir fonksiyonu (B) çağırması ve B fonksiyonun doğrudan veya dolaylı yoldan A fonksiyonunu tekrar çağırması durumudur.

Rekürsif Fonksiyonların Bellek Kullanımı

Bir fonksiyon çalıştırıldığında; o fonksiyonun yerel değişkenleri, parametreleri, dönüş adresi gibi bilgiler **call stack**'e push edilir ve işlem sonlandığında stack'ten pop'lanır.

Rekürsif fonksiyonlarda da fonksiyon kendini her çağırdığında call stack'e push edilir ve base case'e ulaşıldığında sırayla pop edilerek en başa dönülür. Bu şekilde

de **LIFO** uygulanmış olur.

Rekürsif Algoritmaların Temsili

Rekürsif bir algoritmanın zaman karmaşıklığını ifade etmek için **rekürans bağıntıları (recurrence relations)** kullanılır.

Bir rekürans bağıntısı, algoritmanın bir n boyutlu problem için çalışma süresini $T(n)$, daha küçük boyutlu problemler için çalışma süreleri cinsinden tanımlar. Yani bir fonksiyon kendini çağırıyorsa denklemin de kendini çağırıyor olması gerekir.

Rekürsif Bağıntıları İfade Etme- Örnek 1: Faktöriyel

```
int factorial(int n) {  
    if (n <= 1) return 1;           // Temel durum: O(1)  
    else return n * factorial(n - 1); // Rekürsif çağrı + bir çarpma işlemi O(1)  
}
```

Bu algoritma için rekürans bağıntısı şöyle yazılır:

- $T(1) = O(1)$ (veya sabit bir c değeri)
- $T(n) = T(n-1) + O(1)$ (ve $T(n-1)$ 'in çözümü için geçen süre + sabit zamanlı işlem)

Rekürsif Algoritmaları Çözümleme Yöntemleri

Yinelemeli (iterasyon) Metodu

Bu yöntemde, rekürans bağıntısı, bir örüntü yakalanana kadar adım adım açılır (iterasyon) ve sonra bu örüntü toplam formülü ile ifade edilir.

Örnek: $T(n) = T(n-1) + 1$, $T(1) = 1$ bağıntısını çözelim.

Adım 1: $T(n) = T(n-1) + 1$

Adım 2: $T(n-1) = T(n-2) + 1 \Rightarrow T(n) = [T(n-2) + 1] + 1 = T(n-2) + 2$

Adım 3: $T(n-2) = T(n-3) + 1 \Rightarrow T(n) = [T(n-3) + 1] + 2 = T(n-3) + 3$

Örüntü: $T(n) = T(n-k) + k$

$k = n-1$ için $T(1)$ 'e ulaşırız: $T(n) = T(1) + (n-1)$

$T(1) = 1$ olduğundan: $T(n) = 1 + n - 1 = n$

Sonuç: $T(n) = O(n)$

$T(n) = T(\frac{n}{2}) + 1$, $T(1)=1$ bağıntısını yineleme (iterasyon) yöntemi ile çözelim.

$T(n) = T(\frac{n}{2}) + 1$

$T(\frac{n}{2}) = T(\frac{n}{4}) + 1 \Rightarrow T(n) = T(\frac{n}{4}) + 2$

$T(\frac{n}{4}) = T(\frac{n}{8}) + 1 \Rightarrow T(n) = T(\frac{n}{8}) + 3$

k adım sonra: $T(n) = T(\frac{n}{2^k}) + k$

$\frac{n}{2^k} = 1$ olduğunda $k = \log_2 n$ olur.

$T(n) = T(1) + \log_2 n = 1 + \log_2 n$

Sonuç: $T(n) = O(\log n)$

Yerine Koyma (Substitution) Metodu

Substitution metodu, "**Fake it till you make it**" (**Olana kadar taklit et**) mantığıyla çalışır. Önce doğru olduğunu varsayınız, sonra bu varsayımın matematiği bozup bozmadığını test ederiz.

Örnek Soru: $T(n) = T(n-1) + 1$

1. Hedefi Belirleme (Tahmin)

Şunu göstermeliyiz: $T(n) \leq c \cdot n$

Diyoruz ki: "Bence bu işlemin maliyeti en fazla n 'in bir c katı kadardır." Buradaki c sadece sabit bir sayıyı (mesela 1, 2, 5 gibi) temsil eden bir katsayıdır.

2. Varsayım ve Yerine Koyma (Asıl Olay):

Madem $T(n) \leq c \cdot n$ kuralının doğru olduğunu iddia ediyoruz, o zaman bu kural bir önceki adım olan $(n - 1)$ için de doğru olmalıdır.

Ana formülümüzdeki $T(n - 1)$ kısmını çıkarıp, yerine kendi uydurduğumuz $c \cdot (n - 1)$ kuralını koyuyoruz.

Dağıtma işlemini yaparsak:

$$T(n) \leq c \cdot (n - 1) + 1$$

Bunu biraz daha düzenli yazalım:

$$T(n) \leq c \cdot n + (1 - c)$$

3. Pürüzleri Yok Etme (c'yi bulma)

Bizim en baştaki iddiamız neydi? Sonucun sadece ve sadece $T(n) \leq c \cdot n$ olmasıydı. Ama elimizde fazladan bir $(1 - c)$ yükü kaldı. İddiamızın geçerli olması için bu fazlalığın **sıfır veya sıfırdan küçük (negatif)** olması gerekir ki $c \cdot n$ değerini büyütmesin.

Bunun için $(1 - c) \leq 0$ olmalı, yani $c \geq 1$ seçmeliyiz. $\rightarrow c = 1$ seçelim.

Matematik diyor ki: "Eğer c yerine 1 (veya daha büyük bir sayı) yazarsan bu denklem kusursuz çalışır." Görselde c yerine 1 seçiliyor.

1 koyduğumuzda fazlalık $(1 - 1) = 0$ olur ve elimizde tertemiz şu kalır: $T(n) \leq n$

4. Sağlama (Temel Durum)

Son olarak diyoruz ki: "İyi hoş hesapladık da, bu kural en baştaki ilk adım için çalışıyor mu?" En tepedeki bilgiye göre $T(1) = 1$. Bizim kuralımız da n yerine 1 koyduğumuzda 1 sonucunu veriyor. 1, 1'e eşit (veya küçük) olduğu için ilk basamakta sistem çökmedi. Sağlamayı geçtik.

$$n = 1 \text{ için: } T(1) = 1 \leq 1 \cdot 1 = 1 \checkmark$$

Algoritma Paradigmaları

Algoritma paradigması geliştirilecek algoritmanın **nasıl bir strateji ile tasarlanacağını belirleyen üst düzey bir modeldir**. Örneğin;

- Tüm olasılıkları denemek ise **Brute Force** yaklaşımıdır.
- Her adımda en iyi yerel kararı almak **Greedy** yaklaşımıdır.
- Bir problemi daha küçük parçalara ayırarak çözmek **Divide & Conquer** yaklaşımıdır.

1. Brute Force Paradigması

Bir problemi çözmek için **tüm olası çözümleri sistematik olarak deneme** yöntemidir.

Brute Force yaklaşımının avantajı **basit ve uygulanmasının kolay olmasıdır**. Ancak çözüm uzayı büyüdükçe algoritmanın çalışma süresi çok hızlı artabilir. Bu nedenle brute force algoritmalar genellikle **küçük veri setlerinde veya başka yöntemlerin uygulanamadığı durumlarda tercih edilir**.

2. Greedy Paradigması

Her adımda **o anda en iyi görünen seçimi yapma** stratejisini kullanır. Bu yaklaşımda algoritma gelecekteki adımları düşünmeden, yalnızca mevcut durumda en avantajlı seçimi yapar.

Greedy algoritmalar genellikle **çok hızlı çalışır**, çünkü her adımda yalnızca yerel bir karar verilir. Ancak bu yaklaşımın doğru sonuç vermesi için problemin bazı özelliklere sahip olması gerekir:

1. Açgözlü seçim özelliği
2. Optimal altyapı

Bu özellikler sağlanmazsa greedy yaklaşım global optimum sonucu garanti etmeyebilir.

3. Dinamik Programlama Paradigması

Alt problemlerin tekrar tekrar çözüldüğü durumlarda kullanılan bir optimizasyon tekniğidir.

Bu yaklaşımda problem daha küçük alt problemlere ayrılır ve her alt problem **yalnızca bir kez çözülür**. Elde edilen sonuçlar saklanarak tekrar hesaplanması engellenir.

Dinamik programlama 2 önemli özelliğe sahip programlarda kullanılır:

1. **Optimal Substructure:** Bir problemin optimal çözümü, alt problemlerin optimal çözümlerinden oluşur.
2. **Overlapping Subproblems:** Aynı alt problemler birden fazla kez ortaya çıkar.

Örneğin; klasik Fibonacci dizisinde ($Fib(5)$ 'i bulmak için $Fib(4)$ ve $Fib(3)$ 'ü hesaplarsın. Ama $Fib(4)$ 'ü bulmak için de tekrar $Fib(3)$ 'ü hesaplaman gerekir). Dinamik programlama, bu gereksiz tekrarları engellemek için tasarlanmıştır.

4. Divide & Conquer

Divide & Conquer yaklaşımı, büyük bir problemi **daha küçük alt problemlere bölerek çözme** prensibine dayanır.

1. Problem daha küçük parçalara bölünür.
2. Her alt problem ayrı ayrı çözülür.
3. Alt problemlerin çözümleri birleştirilerek ana problem çözülür.

Bu paradigma özellikle rekürsif algoritmaların tasarımında yaygın olarak kullanılır ancak her rekürsif divide and conquer paradigması değildir çünkü birleştirme kısmı olması gerekir.

Örnek algoritmalar: Binary search, merge sort, quick sort...

Paradigma	Temel Fikir	Avantaj	Dezavantaj
Brute Force	Tüm çözümleri denemek	Basit ve genel	Çok yavaş olabilir
Greedy	Her adımda en iyi yerel seçim	Hızlı	Her zaman optimal çözüm vermez
Dynamic Programming	Alt problemlerin sonuçlarını saklamak	Çok verimli çözümler	Bellek maliyeti olabilir
Divide & Conquer	Problemi parçalara bölmek	Büyük problemler için etkili	Rekürsiyon maliyeti

1. Binary Search

- Binary search yalnızca sıralı dizilere uygulanabilir.

- **Zaman Karmaşıklığı:** $O(\log n)$, $\Omega(1)$, $\Theta(\log n)$
- **Uzay Karmaşıklığı:** Döngü ile yazıldığı durumda $O(1)$, rekürsif yazıldığı durumda $O(\log n)$ ek uzay kullanabilir.

2. Merge Sort

- Diziyi alt parçalara böler, her alt parçayı kendi içinde sıralar, sıralı 2 parçayı birleştirerek tek bir sıralı dizi elde edilir.
- Parçaların her birinin eşit elemanlı olabilmesi için main array'in 2^n elemanlı olması gerekir.
- $T(n) = O(n \log n)$, $\Omega(n \log n)$, $\Theta(n \log n)$ 'dir.
- Merge Sort'un dezavantajlarından biri ek bellek ihtiyacıdır. Uzay karmaşıklığı $O(n)$ ek maliyet doğurur.

Bir Algoritmanın Kararlılığı (Stable)

- Eğer bir sıralama algoritması, eşit değerdeki 2 elemanı sıralarken bunların yerini değiştirmiyorsa kararlıdır.

$[(5, A), (3, B), (5, C)]$

Burada iki tane 5 vardır: A ve C.

Sıralama sonrası eğer sonuç:

$[(3, B), (5, A), (5, C)]$

oluyorsa algoritma kararlıdır. Çünkü eşit anahtarlı elemanların sırası korunmuştur.



Merge sort bu yönden **kararlıdır**.

3. Quick Sort

Temel amacı, diziyi bir pivot etrafında iki parçaya bölmek ve bu parçaları ayrı ayrı sıralamaktır.

Quick sort kararlı (stable) bir algoritma değildir.

Quick Sort'un Zaman Karmaşıklığı

Quick sort'un verimliliği pivot elemanın iyi seçilip seçilememesine bağlıdır.

- **En İyi Durum:** En iyi durumda pivot diziyi her seferinde yaklaşık iki eşit parçaya böler.
 - $T(n) = 2T(n/2) + O(n)$, Bu bağıntının çözümü $O(n \log n)$ şeklindedir.
 - Yani en iyi durumda Quick Sort, Merge Sort ile aynı asimptotik performansı verir.
- **Ortalama Durum :** Ortalama durumda da pivotlar genellikle tamamen dengeli olmasa bile çok kötü de değildir.
 - Bu nedenle quick sort'un ortalama zaman karmaşıklığı $O(n \log n)$ kabul edilir.
- **En Kötü Durum:** Pivot elemanın her seferinde en büyük ya da en küçük seçildiği durumdur.
 - Zaman karmaşıklığı $O(n^2)$ olur

Quick Sort'un Uzay Karmaşıklığı

Quick sort in-place bir sıralama algoritması olduğu için yeni dizi oluşturmaz ancak rekürsif işlem yaptığı için ek uzay maliyeti oluşur.

- En iyi durumda $O(\log n)$,
- En kötü durumda ise $O(n)$ olur.

Sıralama Algoritmaları

Karşılaştırmalı Sıralama Algoritmaları

Bu algoritmalar elemanları birbiriyle karşılaştırarak sıralar.

Örnek olarak; **bubble sort**, **selection sort**, **insertion sort** verilebilir.

Bu algoritmalar için teorik alt sınır $O(n \log n)$ 'dir.

Karşılaştırmaz Sıralama

Karşılaştırma yapmak yerine farklı teknikler uygulanır.

Örnek olarak; counting sort, radix sort, bucket sort verilebilir.

Karşılaştırmalıya göre daha hızlı olabilir ($O(n)$)

Ancak, veri sınırı olmalı ve ek bellek gerektirmektedirler.

Basit Sıralama Algoritmaları

1. Insertion Sort

Elemanları sırayla dolaşarak her elemanı en soldaki sıralı kabul edilen kısımdaki uygun yerine koyar.

Stable bir algortimadır.

Insertion Sort'un önemli özelliği, giriş verisinin yapısına duyarlı olmasıdır. Yani veri sıralıya yakınsa algoritma çok daha hızlı çalışır. Bu nedenle "adaptive" bir algoritma olarak değerlendirilir.

Zaman Karmaşıklığı

- **En iyi durumda:** $O(n)$ (dizi zaten sıralıdır)
- **En kötü durumda:** $O(n^2)$ (dizide sürekli en sondan en başa gitmek zorunda kalır ve dizi sürekli komple sağa kaydırılır.)

2. Selection Sort

Dizinin sıralanmamış kısmındaki en küçük elemanı bulup sıralı kısımdaki yerine koyar.

Stable değildir.

Selection sort adaptive değildir çünkü dizi komle sıralı da olsa, ebesini tersten görmüş de olsa dizi baştan sona sürekli taranacaktır.

Zaman Karmaşıklığı

- En iyi durumda da en kötü durumda da diziyi komple tarayacağı için $O(n^2)$ olur

3. Bubble Sort

Stable bir algortimadır.

Zaman Karmaşıklığı

- En iyi durumda swap işlemi olmayacağı için $O(n)$
- En kötü durumda ise $O(n^2)$ kabul edilir.

Farklı Bağıntı Türleri

1. Linear Azalan

- $T(n) = T(n - 1) + n$
- Sonuç: $O(n^2)$

2. Logaritmik Azalan

- $T(n) = T(n/2) + 1$
- Açılım:
- $T(n) = T(n/2) + 1 = T(n/4) + 2 = \dots$
- $T(n) = O(\log n)$

3. Böl-Parçala Türü

- $T(n) = aT(n/b) + f(n)$
- Bu yapı Master Teoremi'nin temelidir

Master Teoremi

Master teoremi şu tip rekürsif bağıntılar için geçerli olan bir yöntemdir:

$$T(n) = aT(n/b) + f(n)$$

a: alt problem sayısı

b: alt problemlerin küçülme oranı

f(n): alt problemler dışında yapılan iş

Bu denkleme göre alt problemlerin toplam maliyeti:

$$n^{\log_b a}$$

Bu durumda Big- Θ bulunmasında 3 durum oluşur:

1. Alt problemler baskınsa, yani f(n) alt problemlerin toplamından daha küçükse $\Theta(n^{\log_b a})$ olur.
2. Eğer ikisi de eşitse $\Theta(n^{\log_b a} \cdot \log n)$ olur
3. Dış maliyet baskısı daha fazlaysa $\Theta(n^{f(n)})$

Algoritma	Kategori	En İyi Zaman (Best)	Ortalama Zaman	En Kötü Zaman (Worst)	Alan (Space) Karmaşıklığı	Kararlı mı? (Stable)
Binary Search	Arama	O(1)	O(logn)	O(logn)	İteratif: O(1) Rekürsif: O(logn)	-
Merge Sort	Sıralama	O(nlogn)	O(nlogn)	O(nlogn)	O(n)	Evet
Insertion Sort	Sıralama	O(n)	O(n ²)	O(n ²)	O(1)	Evet
Bubble Sort	Sıralama	O(n)	O(n ²)	O(n ²)	O(1)	Evet
Selection Sort	Sıralama	O(n ²)	O(n ²)	O(n ²)	O(1)	Hayır
Quick Sort	Sıralama	O(nlogn)	O(nlogn)	O(n ²)	O(logn) ile O(n) arası	Hayır